Implementación de Operaciones con Matrices y Arreglos en Processing

Edgar David Acosta Bernal

Cristian Felipe Ortiz Cucunuba

Alexander Espinoza García

Universidad de Cundinamarca

Ingeniería de sistemas y computación

Programación I

Facatativá

2 de marzo de 2026

Tabla de contenido

Evidencia de ejecución.....**3**

Matrices iniciales (A y B).....3

Arreglos (v1 y v2).....3

Resultados de: producto de matrices, producto matriz-vector, producto…….....…3

punto de arreglos, transpuesta, inversa (o mensaje de no invertible).....2

Explicación breve y clara **4**

Cómo se llenan matrices (aleatorio y conteo) 4

Qué método usaron para la inversa (y cómo validan casos no invertibles) 4

Convención usada para el producto “arreglo por matriz” 6

Evidencia de ejecución

- Matrices iniciales (A y B)

```
Matriz A:
[ 6.0  6.0  3.0  ]
[ 7.0  2.0  5.0  ]
[ 1.0  1.0  2.0  ]
Matriz B:
[ 1.0  2.0  3.0  ]
[ 4.0  5.0  6.0  ]
[ 7.0  8.0  9.0  ]
```

- Arreglos (v1 y v2)

```
Vector 1:
[ 8.0  10.0  10.0  ]
Vector 2:
[ 8.0  8.0  10.0  ]
```

- Resultados de: producto de matrices, producto matriz-vector, producto punto de arreglos, transpuesta, inversa (o mensaje de no invertible)

```
1. Producto de matrices
[ 51.0  66.0  81.0  ]
[ 50.0  64.0  78.0  ]
[ 19.0  23.0  27.0  ]

2. Producto punto de un arreglo por una matriz
[ 138.0  126.0  38.0  ]

4. Producto punto de dos arreglos
Resultado: 244.0

4. Transpuesta de una matriz
[ 1.0  4.0  7.0  ]
[ 2.0  5.0  8.0  ]
[ 3.0  6.0  9.0  ]

5. Inversa de una matriz
No existe inversa porque un pivote es cero.
```

Explicación breve y clara

- Cómo se llenan matrices (aleatorio y conteo)
  Bueno para empezar con esto que trae cosas de algebra lineal, utilizamos lo que son los arreglos bidimensionales y los unidimensionales que en donde los corchetes ([ ]) actúan como un tipo de coordenadas para detectar los datos en la memoria.

  Para el caso de la matrizA el cual este caso es el de aleatorio el cual dice que con números al azar lo que hicimos fue que con la ayuda de la función la cual es la de random nos ayuda a crear diferentes números cada vez que se ejecuta y uno le puede dejar un limite tipo (1,10) para que solo salgan números dentro de este límite.

  Y para la otra matriz que es la matrizB y esat es la de conteo es la que el cual esta se llena con los números, pero en forma de orden tipo 1,2,3,4… que para que esta serviría se añadió una variable tipo contador que nos ayudar que esta vaya sumando +1

```
//Aca empezamos con la segunda ventana la cual es para que las matrices se puedan llenar los datos

//Vamos a iniciar con la matriz A la cual es una funcion que esta es la que esta desordenada y deja numeros al azar
void llenarMatrizA() {
  for (int i = 0; i < tamanoMatriz; i++) { //Para eso vamos a poner ciclos que con estosnos va a permitir que mire las filas es decir que mire que cada una tiene su numero
    for (int j = 0; j < tamanoMatriz; j++) { //Es lo mismo con el anterior pero en este caso es con las columnas
      matrizA[i][j] = int(random(1, 10)); //En este arreglo lo que hacemos es dejar los ciclos anteriores que son el i y j para que despues podamos dejar que deje numeros random del 1 al 9 en todas las filas y columnas
    }
  }
}

//Ahora vamos a seguir con la matriz B que tambien es una funcion que esta es un poco mas sencilla debido a que esta es con los numeros en orden
void llenarMatrizB() {
  float contador = 1; //Aca añadimos una variable que esta es la qeu nos va ayudar a que sume +1 para que esat matriz pueda ir en orden
  for (int i = 0; i < tamanoMatriz; i++) { // Haremos lo mismo que con el anterior un ciclo para que este pueda recorrer la fila
    for (int j = 0; j < tamanoMatriz; j++) { //Y este para que pueda mirar las columnas
      matrizB[i][j] = contador; //Este = nos ayuda a que guarde la suma
      contador = contador + 1; //Y este es el que hace la suma y va avanzando
    }
  }
}

//Y ahora vamos con la ultima funcion que es para que los vectores se puedan llenar de datos
void llenarVectores() {
  for (int i = 0; i < tamanoMatriz; i++) { //Dejamos este ciclo para que este pueda recorrer la columna (en este caso)
    vectorUno[i] = int(random(1, 11)); //En este arreglo es dejalo con el ciclo de i que va a ser la columna para qeu este pueda dejar numeros al azar en el vector 1
    vectorDos[i] = int(random(1, 11)); //En este arreglo es dejalo con el ciclo de i que va a ser la columna para qeu este pueda dejar numeros al azar en el vector 2
  }
}
```

- Qué método usaron para la inversa (y cómo validan casos no invertibles)
  El método que terminamos usando fue el de gauss jordan y pues este consiste en crear otra matriz que quiere decir seleccionar la matriz que uno quiera en este caso escogimos la matrizA y la otra seria que es de 0 y 1 con el mismo tamaño

  Lo que estamos haciendo es que la matrizA que escogimos anteriormente va a pasar a ser una identidad y la otra matriz que es la de 0 y 1 se va a ir transformando en una inversa

Para validar si esta no tiene una inversa lo que va a ser el código es que revisa el numero de la diagonal

Si este revisa y encuentra un 0 este se va a detener y va a avisar en la consola que no hay una inversa

```
// aqui podremos hallar la matriz inverza con el metodo de Gauss Jordan
void matrizInversa() {
  println("              ");
  println("5. Inversa de una matriz");

  // creamos una matriz de (3 filas x 6 columnas)
  float[][] matrizTrabajo = new float[tamanoMatriz][tamanoMatriz*2];
 //aqui se crea una matriz para los resultados
  float[][] matrizResultado = new float[tamanoMatriz][tamanoMatriz];

  // ahora llenamos la matriz trabajo poniendo la identidad en el dalo derecho
  for (int i = 0; i < tamanoMatriz; i++) { // van las filas
    for (int j = 0; j < tamanoMatriz; j++) { //van las columnas
      matrizTrabajo[i][j] = matrizB[i][j]; // pasa a copiar los datos de A a la izquierda

      // en la matriz identidad dibujamos en la diagonal 1
      if (i == j) {
        matrizTrabajo[i][j+tamanoMatriz] = 1; // ponemos un 1 en la diagonal
      } else {
        matrizTrabajo[i][j+tamanoMatriz] = 0; // ponemos 0 en lo demas
      }
    }
  }
  //aca vamos ha hacer que columna por columna i aparezca en la parte izquierda
  for (int i = 0; i < tamanoMatriz; i++) {
    // la variable q denominamos como pivote venedria siendo el numero q esta en la diagonal A
    float pivote = matrizTrabajo[i][i];

    // en este caso si el pivote es 0, la division marca un error y asi la matriz no tiene inversa
    if (pivote == 0) {
      println("No existe inversa porque un pivote es cero.");
      return; // aqui se detiene y se sale
    }

    // se divide toda la fila actual por el pivote para que ese n° se vuelva 1
    for (int j = 0; j < tamanoMatriz*2; j++) {
      matrizTrabajo[i][j] = matrizTrabajo[i][j] / pivote;
    }

    // esto se hace para q tanto como arriba como abajo del 1 haya solo ceros
    for (int k = 0; k < tamanoMatriz; k++) {
      if (k != i) { // no se hace nada ya que es la fila donde acabamos de poner el 1
        float factor = matrizTrabajo[k][i]; // este es el numero q queremos eliminar de las demas fil
        for (int j = 0; j < tamanoMatriz*2; j++) {
          // Restamos a la fila del pivote la fila del pivote y multiplicamos por el factor
          matrizTrabajo[k][j] = matrizTrabajo[k][j] - (factor * matrizTrabajo[i][j]);
        }
      }
    }
  }

//aqui podemos ver como el lado izquierdo se volvio identidad y el derecho inversa
  for (int i = 0; i < tamanoMatriz; i++) {
    for (int j = 0; j < tamanoMatriz; j++) {
      // se toman los valores de la columna 3, 4, 5 y los pasamos a la matiz limpia
      matrizResultado[i][j] = matrizTrabajo[i][j+tamanoMatriz];
    }
  }

  //se deja ver al usuario
  imprimirMatriz(matrizResultado);
}
```

- Convención usada para el producto “arreglo por matriz”

Lo que hicimos para este es multiplicar cada fila de la matiz por el arreglo de la manera mas organizada posible, en este se empieza en la primera fila de la matriz y se multiplica su primer numero por el primero del arreglo, ya con el 2 por el 2 y así seguimos para que todos esos resultados se van sumando en r[] para darnos en un único valor final por cada fila

Este se va repitiendo durante el proceso con las 3 dilas de la matriz, lo que nos ayuda este a la combinación de una tabla y una lista en otro nuevo arreglo, pero este ya con los resultados.

```
//Aqui ya en el 2 punto el cual es Producto punto de un arreglo por una matriz

// horizonatl
void matrizProductoMatrizVector() {
  println("      ");
  println("2. Producto punto de un arreglo por una matriz");

  float[] r = new float[tamanoMatriz]; // se crea para guardar los 3 resultados

  for (int i = 0; i < tamanoMatriz; i++) { // elige fila de toda la matriz   A vamos a estar trabajando
    r[i] = 0; // se pone es 0 para que inicie limpio
    for (int j = 0; j < tamanoMatriz; j++) { // recorre la matriz y el vertor al tiempo
      r[i] = r[i] + (matrizA[i][j] * vectorUno[j]); //suma lo que ya teniamos guardado
    }
  }
  imprimirVector(r); // nos imprime el vector resultante
}
```